

Reviewer Pack — Minimal Local Index of Hodge Decomposition vs ACC⁰ Circuit L...

Entry 31b0cc22e64f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 17:14:05 UTC

Minimal Local Index of Hodge Decomposition vs ACC⁰ Circuit Lower Bound Proven by Sipser Functions

Entry ID: 31b0cc22e64f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 17:14:05 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Decomposition) **Field B** (complexity object): Complexity Theory: ACC⁰ Circuit Complexity

Statement:

[‘For every explicit function f in P with an ACC⁰ circuit lower bound proven using a Sipser function, the local index of its Hodge decomposition is at most 2.’, ‘Consequently, any explicit function in P with a local index of Hodge decomposition greater than 2 cannot be computed by an ACC⁰ circuit with a Sipser function.’, ‘This implies that the minimal violation for an ACC⁰ lower bound using a Sipser function is associated with functions having a local index of Hodge decomposition ≤ 2 .’]

Rationale (proposer’s reasoning):

[‘The Hodge decomposition in algebraic geometry has been used to study the complexity of certain problems, but its connection to ACC⁰ circuit lower bounds remains unexplored. If proven, this conjecture would provide a novel approach to proving lower bounds for ACC⁰ circuits by studying geometric properties.’, ‘Hodge theory provides a rich framework for understanding the geometric and algebraic structure of varieties. By relating the local index of Hodge decomposition to ACC⁰ complexity, we may uncover new structures that separate classes of functions in P .’]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 239bd234d567aa73

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal Local Index of Hodge Decomposition (LH) for an ACC⁰ circuit lower bound proven by Sipser function is ≤ 2 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge Decomposition" AND "ACC⁰ Circuit Complexity" - "local index" AND "Hodge Decomposition" AND "ACC⁰ circuit lower bound" - "Sipser function" AND "lower bound" AND "Hodge Decomposition"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            for j in range(n):
                A[i][j] /= pivot
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
```

```

        C[i][j] += A[i][k] * B[k][j]
    return C

def hodge_decomposition(A):
    m, n = len(A), len(A[0])
    U = [[0] * n for _ in range(m)]
    S = [[0] * n for _ in range(n)]
    Vt = [[0] * m for _ in range(n)]
    for i in range(m):
        U[i][i] = 1
    for j in range(n):
        S[j][j] = A[j][j]
    for k in range(n):
        Vt[k][k] = 1
    return U, S, Vt

def sipser_function(x):
    return x % 2 == 0

def acc0_circuit_size(A, B):
    m, n = len(A), len(B)
    C = matrix_multiplication(A, B)
    rank = 0
    for row in gaussian_elimination(C):
        if any(row):
            rank += 1
    return rank

def local_index_of_hodge_decomposition(U, S, Vt):
    m, n = len(U), len(S)
    lihd = 0
    for i in range(n):
        if U[i][i] != 0:
            lihd += math.log(abs(U[i][i]))
    return lihd

def generate_random_function(n):
    A = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    return A

n = random.choice([5, 10, 15, 20, 30, 40])
f = generate_random_function(n)
U, S, Vt = hodge_decomposition(f)
lihd = local_index_of_hodge_decomposition(U, S, Vt)
acc0_size = acc0_circuit_size(f, f) if sipser_function(lihd) else float('inf')

return {
    "metric_name": "local_index_of_hodge_decomposition",
    "metric_value": lihd,
    "instances_tested": 1,
    "conjecture_holds": lihd <= 2,
    "counterexample": "" if lihd <= 2 else f"Function with LIHD={lihd}"
}

if __name__ == "__main__":
    import sys

```

```

seeds = [int(x) for x in sys.argv[1:]] or [random.randint(100, 999) for _ in range(30)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_lihd = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_lihd} std=0.0 support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_lihd} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"LIHD > 2\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'local_index_of_hodge_decomposition', 'metric_value': 0.0, 'instances_tested': 1}
TRIAL: {'metric_name': 'local_index_of_hodge_decomposition', 'metric_value': 0.0, 'instances_tested': 1}
TRIAL: {'metric_name': 'local_index_of_hodge_decomposition', 'metric_value': 0.0, 'instances_tested': 1}
TRIAL: {'metric_name': 'local_index_of_hodge_decomposition', 'metric_value': 0.0, 'instances_tested': 1}
TRIAL: {'metric_name': 'local_index_of_hodge_decomposition', 'metric_value': 0.0, 'instances_tested': 1}
TRIAL: {'metric_name': 'local_index_of_hodge_decomposition', 'metric_value': 0.0, 'instances_tested': 1}
TRIAL: {'metric_name': 'local_index_of_hodge_decomposition', 'metric_value': 0.0, 'instances_tested': 1}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bb6946c3.py", line 105, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bb6946c3.py", line 89, in run_trial
    acc0_size = acc0_circuit_size(f, f) if sipser_function(lihd) else float('inf')
                ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bb6946c3.py", line 68, in acc0_circuit_size
    for row in gaussian_elimination(C):
                ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bb6946c3.py", line 31, in gaussian_elimination
    A[i][j] /= pivot
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data that could confirm or falsify the conjecture.
| next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15176
2	preregistration	ollama_remote	glm4:latest	0	0	5369
3	novelty	ollama_remote	glm4:latest	0	0	4743
4	novelty	ollama_remote	glm4:latest	0	0	10047
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15711
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8875
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6937
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11818
9	judge	ollama_remote	glm4:latest	0	0	10781

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 89457 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/31b0cc22e64f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/31b0cc22e64f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/31b0cc22e64f.tar.gz (if generated)